# cuButterfly: Unleashing Performance of Butterfly Computation with Hierarchical Hybrid Dataflow

ISCA 2026 Internal Working Draft – Anonymous – Do Not Distribute

***Abstract*—Butterfly computation underlies Fourier, number-theoretic, Hadamard, and zeta transforms. Their shared dependencies expose a common performance problem. Spatial expansion consumes execution resources and live storage, while temporal reuse requires state retention or repeated communication. Their balance depends on the arithmetic, workload, and hardware. We characterize these demands jointly so that organization can be optimized under a target's capacity, bandwidth, and execution constraints.**

**We present CUBUTTERFLY, which derives dependency-closed local regions, their communication and readiness relations, and resource demands from butterfly dependencies. *Hierarchical hybrid dataflow* combines spatial expansion and temporal reuse along stage and data dimensions, repeating the construction across storage scopes. The resulting organization method reuses these relations across arithmetic cores and workloads while evaluating demand against target hardware constraints.**

**An A100 80GB evaluation observes geometric-mean latency ratios of 1.298× and 1.366× over cuFFT for its FP32 and FP64 FFT complete-call cohorts, including adapted and multidimensional requests. An independent cyclic NTT study reaches 1.308× over GPU-NTT across 54 matched 32/64-bit-word requests. Separate fixed-core controls improve complete-call performance by approximately 1.22× with either FFT leaf, with local organization gains also observed for FWHT and NTT. These measurements support reusable organization on the tested workloads. Selected CKKS complete operations improve by 1.131× over the faster of two native execution choices across 36 short-chain requests. Fixed-core controls distinguish organization gains from core substitution, while FP16 and application losses expose limits. Floating-point errors are reported without a fixed acceptance threshold.**

## I. INTRODUCTION

Butterfly computation is a common building block in scientific computing, machine learning, and cryptography. Fourier transforms support fast convolution and long-sequence models [1]–[3]. Number-theoretic transforms provide exact polynomial arithmetic [4], [5], while Hadamard transforms support low-bit model quantization [6]. Learned butterfly factorizations also expose structured linear layers [7]. Despite different numerical contracts, these computations repeatedly combine value pairs along logarithmic stage sequences. Which execution principles carry across their arithmetic and applications?

Mature GPU libraries already optimize both arithmetic and data movement [8]–[11]. Transform generators such as SPIRAL and FFTX also reuse mathematical descriptions to generate implementations [12], [13]. The opportunity is to make their *organization choices* reusable: how should a compatible arithmetic core retain values, communicate changing pair groups, and share finite resources as the workload or target changes? A common method must characterize these tradeoffs while retaining competitive specialized cores.

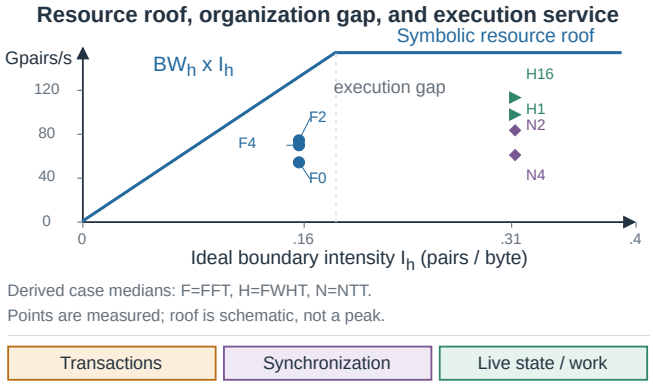



Fig. 1. Measured FFT, FWHT, and NTT organizations attain different rates at equal ideal intensity within each family. The roof is symbolic, not a measured hardware peak.

Arithmetic intensity and Roofline [14] provide a starting point for understanding this variation. Yet matched organizations with the same arithmetic and equal ideal data-reuse intensity attain different pair-update rates on the same device (Figure 1). Organization also changes actual transactions and resource use. We therefore examine **execution organization**: which operations run together, where intermediate values live, and how they reach their consumers. Retention raises ideal reuse but consumes capacity and communication service, and can reduce concurrency. Even abundant parallel work can therefore use hardware inefficiently.

The effect is measurable independently of core replacement. FFT controls improve complete-call performance by about 1.22× while holding each of two arithmetic leaves fixed (Section VI-C). Further FFT, FWHT, and NTT controls show that deeper retention can help or hurt (Section II-B). The question is therefore **whether a common organization method can jointly manage local reuse, communication, and parallel execution across different arithmetic cores, workloads, and hardware.**

CUBUTTERFLY connects these choices through a **dependency-driven resource characterization**. Stage dependencies determine **dependency-closed regions**; intersections between regions determine communication and readiness. Spatial expansion and temporal reuse along the stage and data dimensions then determine live state, communication, core work, and available ready work. Hardware capacities constrain feasibility, and service rates

inform which feasible organizations are promising. The shared structure thus yields workload- and core-specific resource demands.

We use this characterization to construct **hierarchical hybrid dataflow**. Local regions retain values at a selected storage scope and exchange them through explicit handoffs; repeating the construction across scopes handles workloads beyond one local capacity. Boundary layouts account for both producer writes and successor reads, whose changing access groups can otherwise turn reuse into bank contention. Arithmetic, workload, and hardware remain separate inputs, but their performance effects are evaluated jointly. Compatible specialized cores remain part of each realization.

Our two contributions are:

1) A **dependency-driven resource characterization** that derives local regions, communication, and readiness from butterfly dependencies, then connects space–time organization to storage, service demand, and available parallel work.
2) **Hierarchical hybrid dataflow**, which uses these relations to construct local residency and inter-region handoffs under hardware constraints, reusing the construction across compatible arithmetic cores and workload domains.

Subgraph generation realizes these contributions. Library comparisons test competitiveness, fixed-core controls isolate organization effects, and resource measurements test their explanations. Search and calibration support selection within the executable space; they do not guarantee an optimal mapping.

The A100 80GB evaluation reports FFT geometric-mean speedups of 1.298× and 1.366× over cuFFT for its FP32 and FP64 complete-call cohorts, including adaptation and multidimensional requests. An independent cyclic NTT comparison reaches 1.308× over GPU-NTT on 54 matched cases. An independent FWHT comparison gives 1.013–1.031× over DaoFHT with native returns and 1.320–1.330× with supplied-output adaptation. The two output contracts and the fixed-core attribution experiments are reported separately. Section VI details their cohorts, complete applications, and remaining gaps; cross-architecture validation remains open.

## II. Background and Motivation

The organization question begins with the dependency graph. We first identify its recurring structure and numerical variants, then use measured and analytical examples to derive the requirements for a common method.

### A. Butterfly computation and its variants

A butterfly computation combines small groups of values through a sequence of stages. In a radix-two stage, disjoint pairs can execute independently, but the next stage pairs their outputs differently. These changing partners are what make local computation easy to parallelize and the complete computation difficult to organize. For $N = 2^L$, the graph has $L$ stages and $N/2$ pair updates per stage. A stage associated with bit $q$ pairs indices $i$ and $i \oplus 2^q$. The stage sequence defines the dependency envelope, while the pair function supplies arithmetic and coefficients. Batch instances add independent copies of the graph. **Shared structure does not imply interchangeable semantics.**

The family includes complex FFT, finite-field NTT, sum-and-difference FWHT, subset and superset zeta transforms, and products of structured $2 \times 2$ stages. One-sided zeta updates use a subset of the full pairing dependencies. For example, an FFT pair multiplies by a complex twiddle, an NTT pair performs modular arithmetic, and an FWHT pair uses only sums and differences. A common organization must retain these distinctions rather than replace every pair with one arithmetic implementation. Multidimensional separable transforms compose axis transforms with a shared tensor layout, making inter-axis placement part of the same execution problem.

Cooley–Tukey (CT) and Gentleman–Sande (GS) illustrate another distinction [1], [15]. Their local forms place twiddle multiplication on opposite sides of addition and subtraction. Forward and inverse computation also change coefficients, stage order, permutations, and normalization. Stockham algorithms distribute permutations across passes [16]. Section III-D explains which structural relations survive these changes.

The requested precision, direction, normalization, and input/output layout constrain implementation choices. Internal reordering is legal only when it preserves the request, with adaptation included in execution cost.

**The same graph admits different organizations.** Figure 2 holds the 16-point Hadamard graph fixed. Stagewise execution exposes eight independent pairs and transfers all values at every stage. Two-stage groups retain four values each and halve materialized traffic. One whole-transform group minimizes traffic but exposes one independent region per transform. These region counts do not fix worker counts, since workers can cooperate within a region.

Storage scopes give these logical choices physical costs [17]. Worker-local storage, shared storage, and external storage offer different capacities and exchange rates. A generated codelet or imported core such as cuFFTDx [18] must operate within those limits. The preferred grouping therefore depends on both arithmetic service and the surrounding organization.

### B. From a resource roof to an execution organization

Changing pair partners makes data retention and parallel execution interdependent. Roofline bounds their performance opportunity but does not map stages, data, and storage together. We examine the same arithmetic under different organizations to derive that mapping.

**Reuse raises a resource ceiling.** Figure 1 contrasts a symbolic resource roof with measured execution for FFT, FWHT, and NTT. The Roofline opportunity [14] comes from retaining values between dependent stages. For $B$ transforms of length $N = 2^L$, let $e$ be bytes per value and $G$ the number of materialized stage groups. There are $BNL/2$ pair updates. If

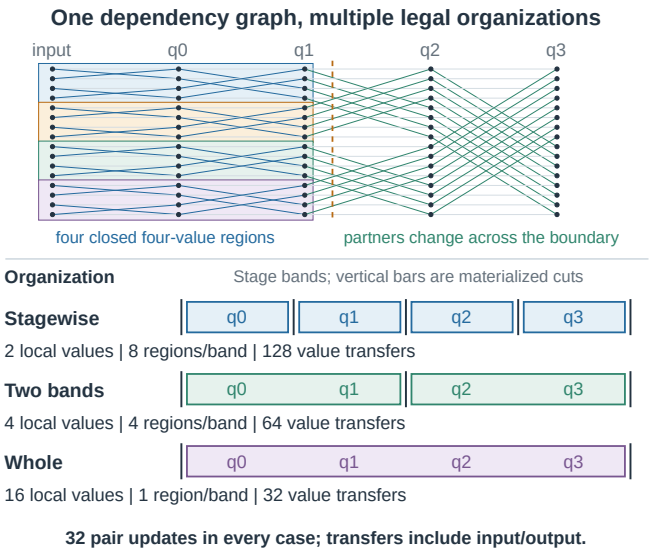



Fig. 2. Grouping stages trades local state and independent regions for fewer transfers in the same Hadamard graph. Counts are analytical.

each group reads and writes the array once, ideal value-only intensity is

$$I_{\text{value}} \simeq \frac{L}{4eG}. \tag{1}$$

At a selected storage boundary, actual intensity is pair work divided by transferred bytes. Its roofline ceiling is the smaller of core throughput and bandwidth times intensity. Coefficients, transactions, cache reuse, and spills change the traffic, while synchronization and execution efficiency determine the distance below this ceiling.

Reducing $G$ raises ideal intensity while changing live state, ownership, communication, and synchronization. The trade-off in Figure 2 therefore affects both the resource ceiling and the execution that approaches it. **The research question is how a parallel organization realizes the available hardware performance.**

**Limiting cases expose different organization choices.** The H1/H16 points use FWHT FP64 at $N = 2^{20}$ and $B = 1/16$ with two materialized bands. Sixteen times as much independent work raises pair rate from 97.8 to 113.4 Gpairs/s at unchanged ideal intensity. Batch can amortize startup and fill idle slots, but cannot by itself resolve an inefficient organization. These timings show limited scaling; traffic and resource measurements are needed to establish its cause.

At $N = 2^{20}, B = 16$, F0/F2/F4 use FFT FP64 with local groups of 0/2/4 stages, and N2/N4 use word64 NTT with groups of 2/4. All retain two materialized bands. Pair rate is $BNL/(2T)$, and ideal intensity follows Eq. 1. FFT reaches 54.4, 74.4, and 70.0 Gpairs/s with the same arithmetic and outer boundary. Local reuse helps, but deeper retention does not continue the gain. NTT instead falls from 83.6 to 61.1 Gpairs/s. Thus equal ideal intensity permits different execution rates, and the preferred local organization depends on arithmetic service and resource use.

Two analytical limits extend this reasoning. If a target doubles the applicable resource roof but a serialized exchange keeps throughput unchanged, efficiency relative to that roof halves. Doubling only arithmetic capacity need not raise a bandwidth-limited roof, so the applicable bound matters. Separately, a larger transform can outgrow local storage despite abundant parallel work, requiring smaller resident tile subgraphs and larger-scope exchanges. These are analytical hardware and capacity cases, not measured cross-GPU efficiency results.

**The trade-off recurs across butterfly families.** The full controls in Figure 11 retain each request's scalar arithmetic primitive and external storage boundaries. Two-stage local grouping improves mean performance for FFT, FWHT, and NTT, while deeper grouping regresses FFT and NTT. Their shared dependencies therefore expose a common trade-off with different preferred choices.

Logical intensity is not measured traffic: grouping also changes exchange, synchronization, and ownership. Matched profiles show increased register use and reduced active warps with deeper grouping. Section VI reports counters and timing ranges without assigning every change to one mechanism. The construction must expose these choices and constraints so that unfavorable allocations can be rejected for each family and workload.

### C. Why a hierarchy and a common construction

The observations call for a **joint characterization of resource demand**. Spatial expansion needs concurrent execution, live state, and communication. Temporal reuse retains or moves values for later work at the chosen storage scope; it need not imply repeated external storage accesses.

Stage and data positions provide two allocation axes. Distinct resources can serve successive stages or independent regions; the same resources can instead be reused over either dimension. Their combination forms **hybrid dataflow**. Characterizing the resulting state, traffic, and parallel work makes the trade-off evaluable under a target's constraints. Stagewise execution, a spatial stage pipeline, and a whole-transform local execution are endpoints of this space when their resource demands fit.

Larger transforms can outgrow local storage or bounded compute groups. **Hierarchy repeats the construction under those limits.** Additional bands or recursive refinement preserve locality and handoff rules without fitting the complete intermediate array on chip. Logical depth and materialized groups remain distinct from physical storage levels; refinement adds no hardware capacity.

More batch provides independent work to amortize startup and cover transfers. Realizing that slack still requires lookahead, finite buffers, and available resources. A successor needs its actual predecessor set, not merely a contiguous prefix of producer output. The method must therefore derive both locality and readiness from the graph.

Stage-bit dependencies separate **semantics, workload, and hardware** inputs: arithmetic supplies local functions and la-

bels; length and batch instantiate work; hardware supplies resources and service rates. The next section derives legal local work, handoffs, and resource demands across bounded storage scopes. Block-level and resident subgraph execution realize them on GPUs.

## III. Hierarchical Hybrid Dataflow

The method connects **dependency structure, resource demand, and hardware constraints**. An **operator contract** specifies arithmetic, coefficients, direction, normalization, precision, and logical input/output order; a **workload domain** specifies shape, batch, placement, strides, and storage layout; **hardware capabilities** specify resources and compatible cores. These inputs jointly determine performance. Figure 3 traces the construction: derive dependency-closed regions and handoffs, characterize spatial/temporal demands, then bind selected tile subgraphs to cores and storage scopes under target capacities and service rates. We develop it through a 16-point Hadamard example.

### A. Dependency-closed regions

Local reuse requires a group containing all values needed by its operations. The stage-bit representation constructs such groups directly. **A stage band determines its local data domain.** For a radix-two transform, choose a consecutive stage band and let $Q$ contain its pairing bits. Holding every other index bit fixed gives

$$\mathcal{I}(a, Q) = \{i \mid i \text{ agrees with } a \text{ outside } Q\}. \tag{2}$$

Every pair update in the band changes only a bit in $Q$ and thus stays inside this region. Assignments to the fixed bits partition the array into independent regions of $2^{|Q|}$ values. The band's computation on one region is a **dependency-closed region**. Closure is local to the band, whose inputs and outputs still connect to other bands.

Any ordered stage partition defines such regions, including one band spanning the complete transform when capacity and core support permit. Spatial expansion factors may also be one. Band width controls local reuse, while the number of regions assigned to a worker controls worker reuse. Arithmetic labels remain attached to the operations. In particular, an FFT band may require global-index-dependent twiddles, so a standalone smaller FFT is not automatically a compatible core.

**A running example.** For a 16-point unnormalized Hadamard transform with stage bits $(0, 1, 2, 3)$, write $i = 4r + c$. Two equal bands give four row subgraphs varying $c$, then four column subgraphs varying $r$. Each uses a four-point Hadamard core. Figure 2 places this two-band partition beside stagewise and whole-transform partitions of the full graph. Region membership follows variable bits rather than original contiguity, allowing a representation chosen for physical stores and reads.

### B. Space–time organization and resource demand

Closure defines independent work without fixing an execution group, launch, or pipeline depth. **Hierarchical hybrid dataflow assigns spatial resources and temporal reuse to stages and data.** Successive stages can reuse a worker or occupy different resources; independent regions can execute concurrently or stream through that worker. This organization repeats within regions and across storage scopes.

TABLE I
Spatial and temporal choices impose different resource demands.

| Choice | Demand to characterize |
|---|---|
| Stage in space | Concurrent cores, inter-stage transport, coefficients, and live pipeline state. |
| Stage in time | Retained state or feedback transfers, synchronization, and reuse of execution resources. |
| Data in space | Concurrent workers and state, input/output bandwidth, ports, and coefficient delivery. |
| Data in time | Sequential tile service, state lifetime, buffer capacity, and fill/drain overhead. |

A worker can retain a small subgraph in local storage, compute groups can use an intermediate shared scope, and larger regions can use external storage handoffs. Refining a parent does not require its entire intermediate array to fit on chip. Hierarchical spatial and temporal mapping has precedents [19]. Here, butterfly closure and intersections determine the legal regions and handoffs.

**Characterization separates demand from capability.** At each scope, stage-space and data-space factors describe concurrent stage positions and independent pair work. Their temporal counterparts describe reuse of those resources. Table I distinguishes their demands at the selected storage and execution scope, with the chosen core and layout. These are logical organization choices; a worker count or compiler unrolling alone does not establish spatial execution.

For example, an ideal fully supplied stage pipeline with $U_s$ stage positions and $U_d$ pair units per position has $U_s U_d$ arithmetic units and consumes $2U_d$ input values per service step. At fixed arithmetic capacity, shifting resources toward stages reduces boundary supply per unit of work while increasing internal transport and pipeline depth. Actual cores, layouts, and resource sharing determine the achievable service rate.

At storage scope $h$, residency requires

$$\max_t \big(\mathrm{live}_h(t) + \mathrm{aux}_h(t)\big) \le C_h, \tag{3}$$

where live values, coefficients, and buffers are counted in bytes. Compiled register allocation and storage granularity further constrain the mapping. A tile subgraph that does not fit can be refined, rescheduled, or materialized at a larger scope. **Value residency differs from worker persistence.** A finite kernel may keep intermediates local, whereas a persistent worker may exchange them through external storage buffers.

Alongside peak live bytes, the characterization records boundary transfers, core work, and independent ready work. Capacity constrains feasibility; bandwidth, core service, and readiness constrain completion time. Feedback retained at an intermediate scope consumes its service rather than another external pass. Dependency analysis supplies exact logical sizes,

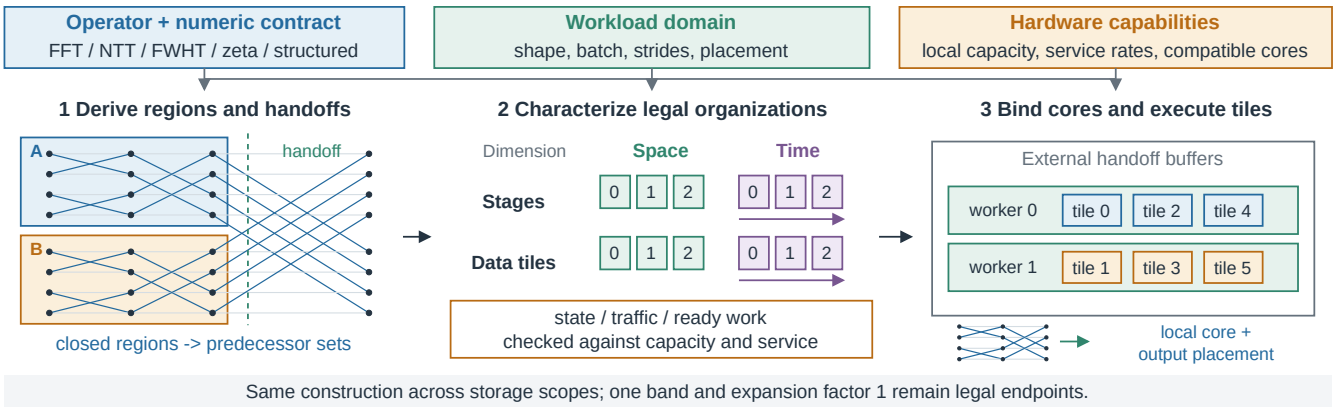



Fig. 3. CUBUTTERFLY maps labeled dependencies through space–time choices to bounded tile execution while retaining specialized arithmetic cores.

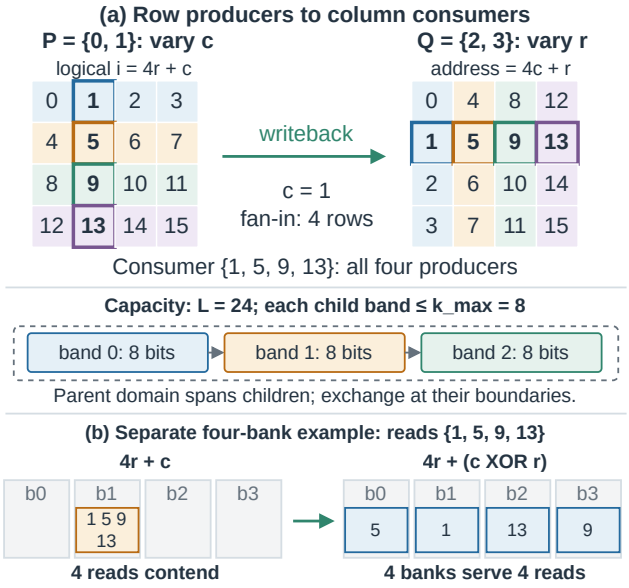



Fig. 4. The Hadamard example exposes dependency, capacity, and bank constraints. (a) Each column needs all row producers. (b) A separate four-bank swizzle distributes its reads; this map is not composed with (a). Counts are analytical.

intersections, and work; lowering and compilation determine physical ownership, allocation, and instructions; measurements supply service rates. Together they distinguish capacity exclusions from performance choices within the executable space. Logical traffic alone does not predict time.

**From dependencies to an organization decision.** A band spanning $s$ distinct pairing bits has $BN/2^s$ independent regions. Retaining all values of one region at the chosen scope needs $e2^s$ bytes, before coefficients, buffering, and core scratch space. If $G$ bands each read and write their values once, they transfer $2eBNG$ value bytes at that boundary. These counts connect a wider band to both greater retention and fewer independent regions. Actual transactions and cooperating threads refine these logical demands.

In the 16-point example, a four-value budget after reserving auxiliary storage permits at most two stages per region retained in local storage. Two bands minimize materialized passes under this capacity-only criterion, transferring 64 values instead of 128 for stagewise execution. A sixteen-value budget permits one band and 32 transfers, but exposes only one region per transform; performance still depends on its core mapping. At the four-value budget, a 32-point transform needs at least three such bands. The construction thus extends beyond two bands without increasing local storage. Feasibility, communication, and parallel organization jointly inform the physical choice.

The construction repeats within regions at successive storage scopes, without requiring every scope or expansion dimension to be active. We first derive regions and boundary relations, then resolve core, layout, and storage demands against capacity. Services and ready work distinguish the feasible choices. The next subsection supplies the boundary rule; Section VI-C follows this construction through measured FWHT and NTT decisions.

### C. Communication and readiness

At a band boundary, changing pair partners connect closed regions through their index-set intersections. A producer and consumer with variable-bit sets $P$ and $Q$ share values when they agree on all bits fixed by both. Their shared variable bits $P \cap Q$ determine each nonempty intersection. Producers covering the remaining consumer bits $Q \setminus P$ form its predecessor set. This derives communication without enumerating scalar edges.

Fixing the bits outside $P \cup Q$ groups connected regions into a **boundary packet**. No edge crosses packets at that boundary. Within a packet, every producer and consumer intersect on the full pairing envelope. For complete-region publication, a consumer must therefore await every producer in its predecessor set. One-sided operators or finer publication units can expose finer dependencies.

In the example, a completed row cannot release a complete column. Independent transforms can nevertheless advance concurrently, and larger transforms may contain multiple packets within one transform. Packet membership is derived at each boundary, not assumed to form a fixed pipeline across the

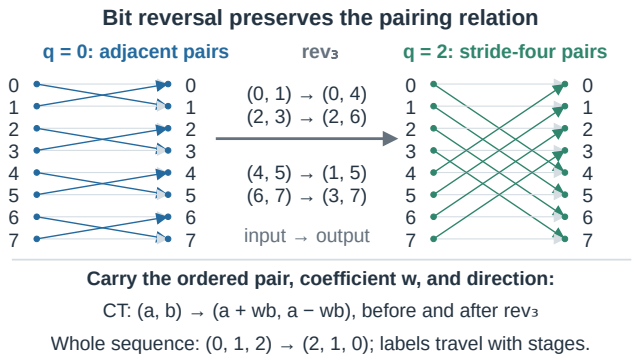



Fig. 5. Bit reversal maps adjacent pairs to strided pairs while preserving their arithmetic labels.

graph. A packet is a dependency domain, not a storage allocation or execution group. Its values can be distributed among workers and external storage buffers. This ownership description does not guarantee conflict-free service. Changing partners can concentrate successor reads on a few storage banks. Boundary placement must account for both producer and consumer access groups (Section IV-B).

### D. Reuse across variants and dimensions

Reuse across stage orders, arithmetic, and tensor interpretations transports the labeled dependency relation, not another transform's numerical output. **Structural correspondence transports organizations.** Bit reversal maps an increasing-stride pairing envelope to a decreasing-stride one, preserving region and boundary relations. Physical layouts must still be evaluated for the transported access pattern.

Figure 5 shows this correspondence on eight indices. Reversing the three index bits sends adjacent pairs to stride-four pairs. Applying the same relabeling to every stage transports the complete envelope and its closed regions. It does not permit stages with different arithmetic labels to be exchanged arbitrarily.

Arithmetic labels move separately. A CT pair computes $(a+wb, a-wb)$ and a GS pair computes $(a+b, w(a-b))$. A GS pair with reciprocal twiddle and one-half scaling inverts the CT pair where those inverses exist. A full inverse also reverses stage order and applies the corresponding permutations. Hadamard changes the local labels to sums and differences, zeta/Möbius exchanges one-sided addition and subtraction, and structured stages require invertible local matrices for inversion. Numerical contracts remain explicit.

For separable power-of-two tensor axes, pairing bits can be labeled by axis and position. Axis-stage composition then uses the same closure and boundary rules on the tensor index domain. Axis order and inter-axis layout become mapping choices subject to the operator's factorization. This describes a shared construction, not a claim that every multidimensional or arithmetic variant already has an efficient lowering.

### E. Correct composition

The arithmetic, placement, and scheduling rules must compose into the requested complete operator. A composed execution is correct when it covers each required operation once, each core implements its labels, every handoff preserves value identity, and every read occurs after production and before storage reuse. A topological induction suffices. Correct predecessor values reach the next operation through consistent layouts and valid lifetimes, and its arithmetic produces the required outputs. Thus local arithmetic, boundary layout, and scheduling carry distinct correctness obligations. The next section realizes these obligations with group-level and resident subgraph execution.

## IV. Realizing the Method with Subgraph Execution

Lowering binds closed regions and boundary dependencies to GPU cores, storage, and synchronization, then generates kernels and a submission graph. A core declares logical value roles, worker ownership, and output visibility, not only input size. Local codelets, compute groups, generated kernels, and imported cores can share the construction with different resource costs. For scalar radix-2 FFT, FWHT, and NTT, common execution templates implement indexing, traversal, publication, and buffer exchange. Arithmetic traits supply pair operations, coefficients, and scaling. Specialized cores retain their numerical restrictions and may require different boundary realizations.

### A. Successor-aligned writeback

A locally efficient producer may leave outputs inconveniently arranged for its consumer. Rather than add a permutation pass, it can incorporate the next placement into writeback when ownership and stores permit. For each value $i$, the producer's output and successor's input representations must agree,

$$\lambda_{\text{out}}(i) = \lambda_{\text{in}}(i). \tag{4}$$

Together with compatible ownership and publication, this defines **successor-aligned writeback**.

One construction moves the successor's variable stage bits into low-order address positions, placing its region in a contiguous range. In the Hadamard example, a producer can write logical index $4r + c$ at address $4c + r$, so the next band reads a column contiguously. Identity placement and explicit permutation remain alternatives. Consumer locality alone does not guarantee efficient stores, so the candidate cost includes both transactions and any staging.

### B. Resolving boundary bank conflicts

**An aligned layout can still serialize its consumers.** Successor tiles gather different combinations than producers write. Distinct concurrent addresses can map to the same bank, whose limited ports serialize service even when values fit and successor reads are aligned.

We address this boundary mismatch with a **successor-aware bank mapping**. The producer and consumer use the same physical placement for each value. Candidate maps combine stage-bit placement with bijective address remapping, including XOR-based *swizzles*. Algorithm 1 gives a bank/offset

construction for an aligned contiguous write group and a strided read group. It expresses the mapping in storage-bank coordinates, so the physical address encoding remains specific to the storage scope.

**Algorithm 1. Map a producer–consumer boundary.**

**Given** $N$ words, $M = 2^k$ banks, stride $S = 2^s$,
$M \le S \le N$ and power-of-two $N$.
**For each** logical word $i \in [0, N)$
$b \leftarrow (i \oplus (i \gg s)) \,\&\, (M-1)$
$o \leftarrow i \gg k$
placement$[i] \leftarrow (b, o)$
**Use** this placement for producer stores and consumer loads.

For an aligned write group $i = aM + u$, $0 \le u < M$, the high-bit term is constant, so the bank is $u$ XOR a constant. For a read group $i = c + vS$, $0 \le c < S$ and $0 \le v < M$, the bank is $(c \bmod M) \oplus v$. Both are permutations of the $M$ banks for full groups; valid tails remain distinct. Within each offset, the low-bit mapping is also a permutation, giving a bijection over the storage domain. This establishes conflict freedom for these single-word groups on single-ported banks, rather than for arbitrary concurrent accesses.

For an illustrative square tile with power-of-two side $P$, $P$ single-ported banks, and one word per value, consider

$$\lambda(r, c) = P\,r + (c \oplus r). \tag{5}$$

With bank index $\lambda \bmod P$, linear placement $Pr + c$ sends a whole column to one bank. XOR by a fixed row or column is a permutation, so the map above distributes either access across all $P$ banks. In Figure 4b, consumer values $\{1, 5, 9, 13\}$ move from bank indices $(1, 1, 1, 1)$ to $(1, 0, 3, 2)$ without a separate data-reordering pass. On GPUs, such address remapping is commonly called a shared-memory bank swizzle [20].

Choose a common address map for the producer's write group and successor's read group. Conflict analysis uses actual transactions, value width, and bank capacity; vector values and broadcasts require separate checks. Outer-storage layouts instead require alignment, transaction-count, and coalescing checks. Thus layout and ownership are chosen jointly; one XOR formula need not suit every core or storage scope.

### C. Residency, publication, and finite buffers

Correct placement still needs readiness and safe reuse. A resident tile subgraph tracks live values and unfinished consumers; an external handoff publishes a dependency unit. Successors start only after their predecessor sets are ready. A buffer slot is acquired before writing, published after production, and released after its final required read. More slots trade lookahead against live state and synchronization. Tail tiles retain their shape and release event.

**Within-band pipelining and cross-band overlap are separate choices.** Figure 6 shows cross-band readiness at the top: one completed Hadamard row leaves every column waiting for three others. After the full row band publishes, columns can overlap another transform's rows if resources permit; the buffer remains live through the final read. The bottom shows within-band load/compute/store overlap using two slots, which

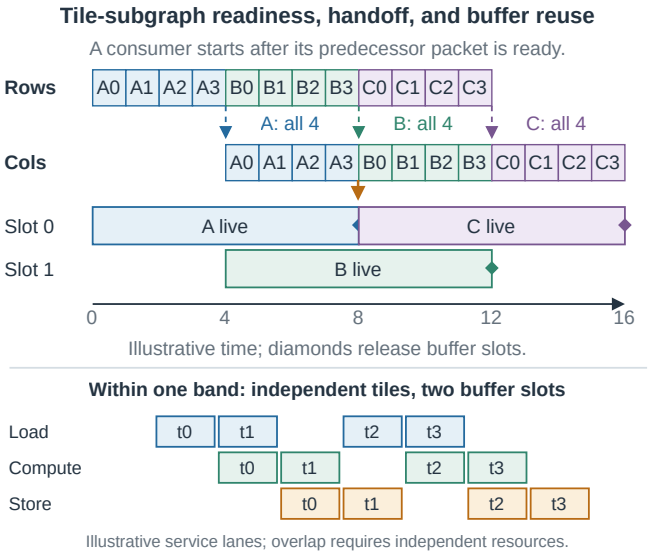



Fig. 6. Top: cross-band readiness for independent Hadamard transforms; C reuses slot 0 after A's final read. Bottom: within-band load/compute/store pipelining with two slots. Durations and lanes are illustrative.

can operate despite a barrier before the next band. Cross-band overlap additionally needs ready predecessors and a free buffer. Batch supplies work for both, subject to lowering, sharing, and lifetime constraints.

### D. Composing physical services

Placement and lifetime translate resource demand into completion time. A **service model** separates the startup, repeated work, and tail of a stage band's execution, avoiding the missing boundary costs of a sum of isolated core times,

$$T_{\text{group}} = T_{\text{start}} + R\,T_{\text{step}} + T_{\text{tail}}. \tag{6}$$

The terms depend on the core, tile shape, layout, launch geometry, and storage scope. A complete-kernel measurement already includes its launch and tail. Bulk execution sums stage-band services plus boundary costs not already included.

For $K$ tiles in a repeatable pipeline, the corresponding view is

$$T_{\text{pipe}} \approx T_{\text{fill}} + (K-1)\,\text{interval} + T_{\text{drain}}. \tag{7}$$

Ready dependencies, finite-buffer reuse, and shared execution or memory resources limit the interval. Overlapped groups need not retain their isolated service rates. More independent work amortizes fill and drain and can cover transfer latency, but cannot exceed a saturated resource's throughput. The model ranks realizations under these assumptions rather than proving optimality.

## V. Kernel Generation and Selection

The workload-dependent organization trade-off motivates selection even at fixed arithmetic. The inputs in Figure 3 feed the preparation and execution paths in Figure 7, instantiating subgraphs and handoffs. Storage constraints prune infeasible realizations; services and readiness rank the remainder. Residency, layout, and concurrency are chosen jointly, including

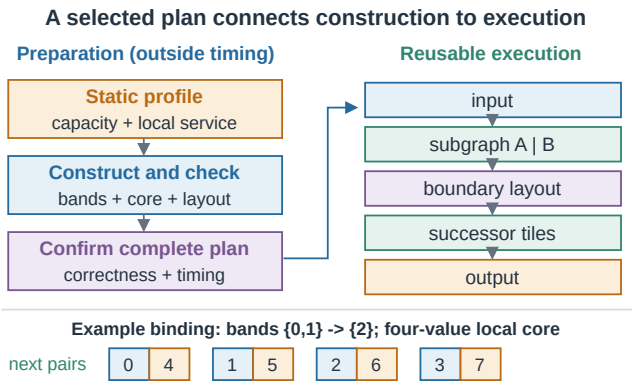



Fig. 7. Preparation constructs and checks subgraph plans, then the selected plan controls local execution and boundary exchange.

handoff costs. Preparation produces a reusable plan whose complete device path is the unit of warm execution.

### A. Generate compatible realizations

Generation combines stage bands and closed regions with compatible cores, storage, boundary layouts, and bulk or pipelined schedules. These coupled choices preserve the requested arithmetic, precision, direction, normalization, order, shape, and placement. Adapted plans include conversion costs. A legal organization without a compatible lowering remains a gap.

Static checks enforce dependencies and capacity; compilation exposes local allocation, intermediate-capacity, temporary-storage, and launch requirements. Numerical checks precede timing. Cache reuse requires matching semantic, workload, hardware, and compilation assumptions.

### B. Estimate local services and their composition

To avoid remeasuring shared local work, services are keyed by arithmetic, core, tile shape, layout, and resource regime. Full and remainder tiles remain distinct when their work differs. Piecewise linear interpolation estimates nearby loads only within a compatible measured regime; missing coverage requires a local measurement or an explicit gap.

Composition follows the actual graph, admitting a group after predecessors and buffers are ready. Shared execution and memory capacity can lengthen concurrent service intervals. Submission costs are added only when absent from measured services, separating local work from contention and orchestration. These estimates guide preparation; the research contribution is the common organization characterization and its realization under hardware constraints. Search is the engineering mechanism that instantiates this space, not a claim of a new global optimization algorithm.

### C. Bounded search and confirmation

Selection compares complete realizations under a finite measurement budget. Compatible cached plans and calibrated services avoid repeated preparation. Measurement modes use bounded enumeration or model-guided exploration; explicit evolutionary tuning adds multistart neighborhood mutation. Multidimensional selection measures complete candidates before refining axis choices. These strategies share semantic and resource constraints, while retaining the complete cost of each organization. Failed checks are excluded from timing and retained in the experiment record.

Complete-plan timing is the unit of confirmation, and static rankings remain distinguishable from measured selection. Organization comparisons distinguish explicit interventions from searched plans and retain their actual mappings, tuning conditions, and confirmation protocol. Preparation is separate from warm execution. The evaluation tests resulting performance and organization effects; finite search does not establish global optimality.

## VI. Evaluation

We ask whether the common construction produces competitive operators and useful application gains, whether organization contributes with a fixed arithmetic core, and which resource costs explain its benefits and limits. External comparisons establish practical performance; controlled experiments isolate organization choices. Their populations and timing boundaries remain separate.

### A. Experimental setup

**Platforms and workloads.** The main comparison uses A100 80GB PCIe (SM80), CUDA 12.4, and PyTorch 2.5.1. Its 66 requests cover FFT, NTT, FWHT, subset/superset zeta, and structured butterflies at ranks 1, 2, 3, 4, and 8. Precision, direction, normalization, order, shape, batch, placement, and strides define a request. An independent cyclic NTT study covers 90 requests with 32/64-bit words, multiple moduli, both directions, $N \in \{256, 65536, 1048576\}$ and $B \in \{1, 7, 16\}$. Nine public FP16 selections span $N = 2^{12}$–$2^{18}$ and $B = 4$–2048. Further BF16 comparisons are deferred; cuFFT Xt's BF16 storage and mixed internal arithmetic differ from our native-BF16 core.

Historical local-grouping, cross-core, and low-precision boundary controls use A100 40GB; the independent service and tuned-endpoint studies use 80GB. Fixed-core FFT controls use 80GB, with an independent 40GB FP32 replication. Each cohort retains its device, build, and protocol identity; subsequent refinements do not replace earlier measurements. All reported platforms are A100. Cross-architecture evaluation on PRO 6000 remains unmeasured.

**Numerical and timing contracts.** All 29 main-study integer and 90 cyclic NTT requests agree exactly with their references. The 37 main-study floating requests have finite outputs and recorded errors without a fixed acceptance threshold; their performance results are exploratory. Floating controls retain their stated reference precision and errors. FP16 scale controls repeat one seeded transform, scaled by $2^{-6}$, across batches and backends rather than providing independent numerical samples. Random weights and short-chain cryptography establish neither task quality nor production security.

Operator timing includes required conversions and output materialization. After ten warmups, each applicable baseline

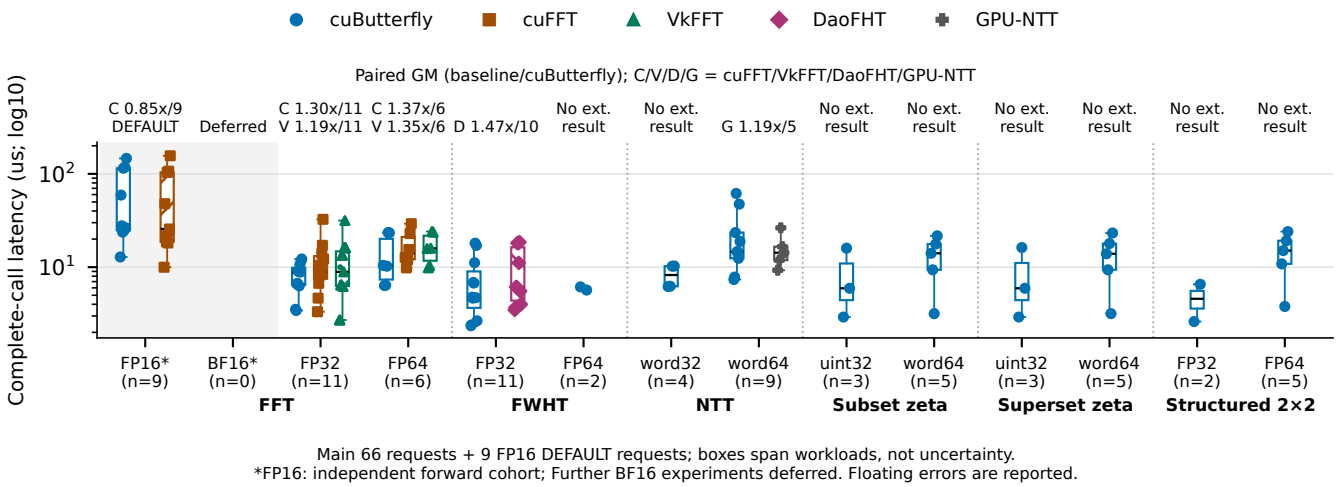



Fig. 8. Frozen complete-call comparisons span operator families and precisions. The main-study DaoFHT calls include supplied-output copying; the independent FWHT interface study is reported separately in Table III. FP16 uses nine public selections, while further BF16 comparisons are deferred.

TABLE II
OPERATOR RATIOS RETAIN THEIR MATCHED POPULATIONS. N/A/X COUNTS NATIVE, ADAPTED, AND AXIS-COMPOSED BASELINE CALLS.

| Operator | Baseline | Pairs | GM | Faster | N/A/X |
|---|---|---|---|---|---|
| *Main 66-request study* | | | | | |
| FFT FP32 | cuFFT | 11 | 1.298× | 9/11 | 5/4/2 |
| FFT FP32 | VkFFT | 11 | 1.190× | 5/11 | 9/0/2 |
| FFT FP64 | cuFFT | 6 | 1.366× | 5/6 | 3/3/0 |
| FFT FP64 | VkFFT | 6 | 1.354× | 6/6 | 6/0/0 |
| FWHT FP32 | DaoFHT | 10 | 1.474× | 9/10 | 0/10/0 |
| NTT w64 | GPU-NTT | 5 | 1.188× | 4/5 | 0/5/0 |
| *Independent FP16 public DEFAULT study* | | | | | |
| FFT FP16 | cuFFT | 9 | 0.854× | 1/9 | – |
| FFT BF16 | cuFFT | 0 | – | – | – |
| *Independent cyclic NTT study (90 requests)* | | | | | |
| NTT w32 | GPU-NTT | 18 | 1.400× | 18/18 | – |
| NTT w64 | GPU-NTT | 36 | 1.265× | 33/36 | – |

captures twenty calls per CUDA graph and reports the median of seven event-timed replays divided by twenty. Search, compilation, allocation, capture, and checking are outside this warm execution boundary. Application ratios match boundaries within each cohort: FNO and QuaRot use host-wall medians, filtering and Poisson use device-event medians, and CKKS/bootstrap use device-event sample means. We do not pool these application protocols. Profiler timings do not replace ordinary timings; Graph replays are not independent process trials, and reported ranges are not confidence intervals. Near-parity ratios do not establish significance.

### B. Performance comparisons

**Question 1: How competitive are the generated operators?** Figure 8 and Table II report matched baseline/CUBUTTERFLY latency ratios. The main 66 requests yield 49 external pairs, with multiple competitors for some requests and no matched baseline for others. Boxes span workloads, not uncertainty. Missing pairs are not counted as wins.

TABLE III
FWHT GAINS DEPEND ON THE MATCHED OUTPUT INTERFACE. RANGES SPAN THREE PROCESS GMS; DEFAULT IS A SEPARATE SINGLE-PROCESS CHECK.

| Output interface | Fixed-mapping replays | Public DEFAULT |
|---|---|---|
| Native return | 1.013–1.031× | 1.009× |
| Supplied buffer | 1.320–1.330× | 1.324× |

FFT FP32 and FP64 achieve geometric means (GM) of 1.298× and 1.366× over cuFFT. These are complete-call gains, including requested layout and normalization adaptation. Native cuFFT subsets give 1.034× over five FP32 pairs and 1.225× over three FP64 pairs; adapted subsets give 1.290× over four FP32 pairs and 1.524× over three FP64 pairs. Independent rank-4/8 axis-composition replays retain 1.925/2.748×. The gains therefore combine local execution and organization around the cores.

**Output contracts affect the FWHT comparison.** The main-study 1.474× against DaoFHT includes copying its returned tensor to a supplied buffer. Table III instead shows an independent study of both interfaces after contiguous-access specialization, using the same ten requests and selected mappings. Both sides retain twenty independent outputs in the native-return Graph; both reuse one caller buffer in the supplied-output Graph. Each comparison matches output lifetime, but subtracting the two interfaces would not isolate copy cost.

The public DEFAULT check selects the same ten mappings. All sixty paired outputs before and after specialization are byte-identical; independent CPU FP32 errors and historical FP64-reference diagnostics are retained separately. Native-return results are near parity, with individual losses, while the supplied-output interface benefits more. FP64 FWHT, zeta, and structured butterflies lack matched GPU baselines. Their separate 34-request CPU study uses two processes on a shared host and a different timing boundary.

**Finite-field range and batch.** The five main-study word64

NTT pairs give 1.188×; three-trial replay gives 1.175×. The independent cyclic study obtains 54 exact GPU-NTT pairs with GM 1.308×, including 51 ratios above one. Its 18 word32 pairs give 1.400× and 36 word64 pairs give 1.265×. Comparator modulus limits ($2^{30}$ for word32 and $2^{61}$ for word64) exclude 36 requests, which retain exact CUBUTTERFLY results without external ratios. Three word64 $B = 1$ pairs yield 0.972–0.999×. Eighteen pairs were measured together; 36 pair unchanged CUBUTTERFLY timings with separately collected baseline timings on the same GPU and protocol.

**Limits of the aggregate gains.** Independent three-process confirmations yield 0.838× for FP64 FFT at $N = 2^{20}, B = 16$ and 0.939× for FP32 FWHT at $N = 2^{15}, B = 32$ on their recorded builds. The nine-request FP16 cohort yields 0.854× against cuFFT, ranging from 0.733 to 1.067×. Only $N = 2^{18}, B = 32$ wins, at 146.53 versus 156.31 $\mu$s. DEFAULT and bounded measured selection choose the same mappings in all nine requests. An independent eleven-request 40GB FP16 cohort gives installed-default GM 0.858× (0.739–1.088×), outside the 80GB aggregate. Its missing independent stage calibration leaves unseen-request ranking unvalidated. These losses coexist with aggregate gains and limit claims of uniform superiority. cuFFT exposes matching storage semantics, not its internal arithmetic.

**Question 2: Do operator gains reach complete applications?** Applications share inputs, precision, keys, and weights as applicable. Complete Graph calls include surrounding computation, packing, conversions, and required copies; reusable setup stays outside timing [21]. Figure 9 retains independent and historical cohorts separately, including losses.

FP32 periodic Poisson at $(N, B) = (1024, 4), (2048, 4), (2048, 16)$ gives 1.082–1.107× over Torch/cuFFT and 1.097–1.125× over CuPy. Twenty event samples include input refresh, both transforms, frequency multiplication, and output copies. Changed/restored-input and output-lifetime checks pass; CPU-reference relative $L_2$ error is below $5.7 \times 10^{-7}$. Fixed $31^2$ filtering on $1024^2/2048^2$ images, $B = 16$, caches the spectrum and matches $1080^2/2100^2$ padding. Two independent repeat processes give 1.068–1.069/1.048× over CuPy. Recomputing the spectrum is a different contract and wins only three of nine earlier requests.

CKKS crosses key switching, square/relinearize/rescale, square/rescale/key switching, and rotation with $B = 1, 8, 32$: methods I/II at $N = 16384$ and method I at $N = 65536$. After loop/stage-batch selection, twenty holdout event-timed rounds of ten Graph replays measure each request. All 36 selected paths produce identical ciphertexts, with GM 1.428× over the native loop. This comparison includes integration gains, which the control below separates. Small-parameter regular bootstrap at $N = 4096$, $B = 1/8$ gives approximately 1.03× over HEonGPU from five samples per backend. Both cryptographic cohorts use `sec_level=none`.

FNO at $64^2/128^2$, $B = 4$, gives 1.030/1.056× in Graph mode and 0.955/0.957× in eager mode, including copies and synchronization [22]. QuaRot's composed conversion/FWHT/scaling path gives 0.957–0.968× at prefill/decode 128/8, $B = 1/4$. An existing core that reads FP16, computes and scales in FP32, and writes FP16 gives 1.004–1.005×/1.015–1.017× for the complete 7B model in both process orders; all nine output steps match exactly [23]. Core selection also changes, so these small gains cannot be attributed to fusion alone. Earlier QuaRot traces without valid random-state preparation and output checks are excluded. The evidence supports application benefits under stated execution contracts, not universal end-to-end speedups.

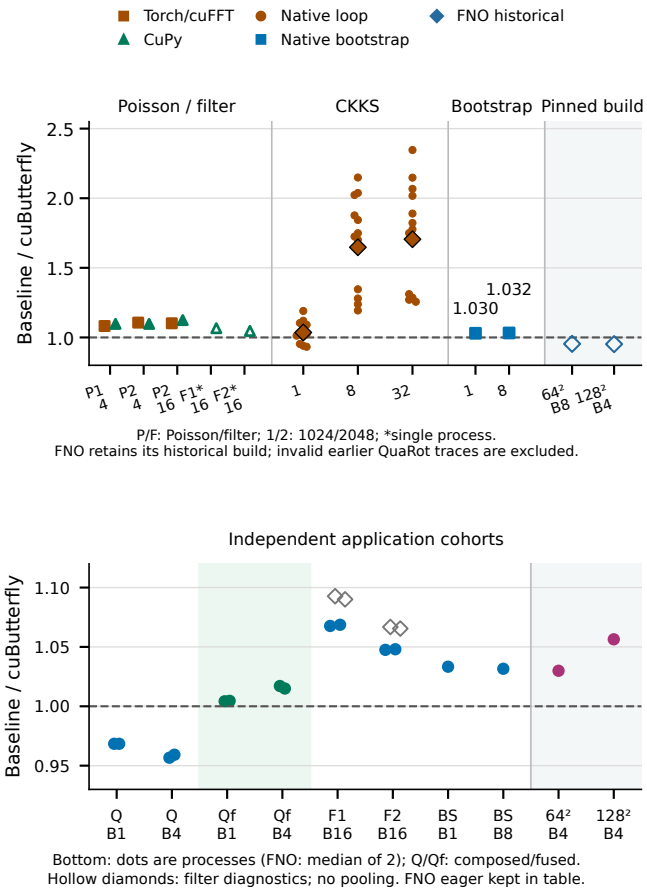



Fig. 9. Application gains vary with workload and integration. The upper panel retains the main and historical cohorts; the lower panel shows independent repeats and composed/fused QuaRot paths. Cohorts are not pooled.

TABLE IV
CKKS GAINS PERSIST AFTER MATCHING BATCH INTEGRATION.

| $B$ | Requests | External Loop | Integration controls Batch | Faster native |
|---|---|---|---|---|
| 1 | 12 | 1.036 | 1.290 | 1.036 |
| 8 | 12 | 1.648 | 1.183 | 1.182 |
| 32 | 12 | 1.705 | 1.182 | 1.182 |

**Matched application integration.** The CKKS control gives native NTT cores the same gather/scatter and Graph organization as our stage-batch path, without assuming an upstream batch API. Table IV compares the native loop, matched batch, and the faster native alternative per request.

Across the same 36 requests, GM falls to 1.218× over

matched native batch and 1.131× over the faster native alternative. Thirty-three ratios exceed one; the three losses occur at $B = 1$, while both larger batch groups retain about 1.182×. This is an integration control, not fixed-core attribution.

### C. Ablations

**Question 3: What does organization contribute with a fixed core?** We first hold the arithmetic leaf fixed and vary execution organization, then test whether gains survive stronger conventional endpoints. Figure 10a crosses register and cuFFTDx Thread 16-point FFT leaves with stagewise exchange and a resident two-stage prefix, sharing later stages. The four cells separate organization changes from core substitution.

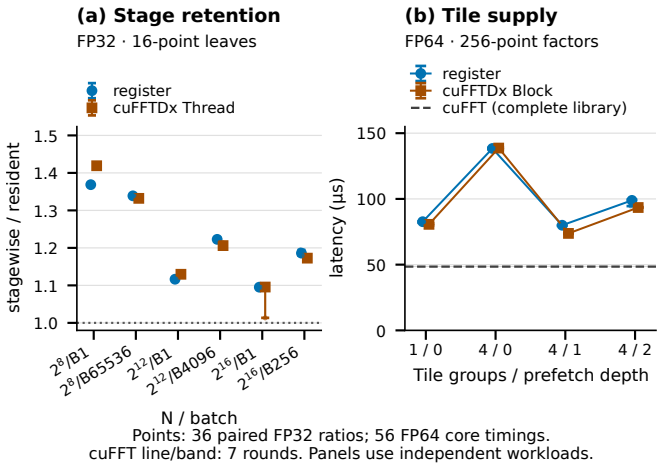



Fig. 10. Fixed-core controls distinguish organization gains from core replacement. (a) Thirty-six paired FP32 ratios come from 72 timings. (b) Points show 56 local-core timings; the cuFFT line and band summarize seven full-library rounds.

Six FP32 workloads, four cells, and three trials produce 72 checked rows, with 10,000 warmups and 100 repetitions. Residency improves all six workloads for both leaves, giving GM 1.217/1.221×, versus 1.042/1.046× from core substitution at fixed organization. Independent replication gives organization gains of 1.226/1.231×. The intervention changes ownership, storage, and launches, not arithmetic; for example, the $N = 256, B = 1$ organizations launch 16 and 256 threads.

**A stronger endpoint bounds the organization gain.** The FP64 control at $N = 65536, B = 16$ fixes two 256-point factors, four local transforms per compute group, and 16 elements per thread. Both cores share the factor template and writer-aligned handoff. At four tile groups per compute group, one-step prefetch reduces latency from 138.377 to 79.974 $\mu$s for the register core and from 138.684 to 73.762 $\mu$s for cuFFTDx Block, or 1.730/1.880×. Against the faster one-tile synchronous alternatives (82.671/80.657 $\mu$s), the gains are 1.034/1.093×. Depth two regresses for both cores; cuFFT remains faster at 48.503 $\mu$s. Relative $L_2$ error against CPU FP64 is below $7.4 \times 10^{-16}$. Thus organization helps either core, and a stronger measured reference substantially reduces the gain.

TABLE V
THE COMMON CONSTRUCTION ADMITS DIFFERENT MEASURED CHOICES. TIMES AND PAIRED RATIOS ARE THREE-PROCESS MEDIANS; "SAME" REUSES ONE MAPPING.

| Leaf | Conv. $\mu$s | Ext. $\mu$s | Ratio |
|---|---|---|---|
| FWHT r2 | 60.447 | 45.875 | 1.318 |
| FWHT r4 | 59.894 | 43.878 | 1.365 |
| NTT shared-r4 | 1585.398 | same | 1.000 |
| NTT warp-r2 | 3706.778 | same | 1.000 |

**Reuse across arithmetic families.** Figure 11 fixes scalar radix-2 FFT/FWHT FP64 and word64 NTT while varying local stage grouping. It covers 18 workloads at $N = 2^{16}, 2^{20}$ and $B = 1, 4, 16$, three organizations, and three process trials, for 162 observations. Two-stage groups give GM 1.196/1.106/1.199× for FFT/FWHT/NTT; groups of up to four stages give 0.932/1.103/0.816×. The shared trade-off admits different preferred organizations rather than a universally beneficial mechanism.

**One construction, different organization choices.** The same dependency rule yields 4096 regions per eight-stage band for FWHT FP32 at $N = 2^{16}, B = 16$, and 16,384 per ten-stage band for word64 NTT at $N = 2^{20}, B = 16$. FWHT regions contain 256 values (1 KiB), and NTT regions contain 1024 (8 KiB). Each uses two bands with disjoint pairing bits. Within a transform, each producer–consumer pair shares one value, so every consumer needs all producers of that transform. This common structure admits different mappings.

FWHT groups one/two/four suffix regions per compute group. Suffix group counts fall from 4096 to 2048/1024, while source-derived shared-storage lower bounds rise from 1 to 2/4 KiB. NTT instead traverses one/two/four independent transforms per group, using one cooperative launch and external handoffs. Traversal grouping does not merge dependency packets. Logical value traffic stays at 16/512 MiB for FWHT/NTT across these choices; neither quantity is measured traffic or simultaneous on-chip storage.

Each fixed leaf retains its conventional endpoint. FWHT tunes 128/256 threads in both domains; NTT fixes 256. Conventional/extended pools contain 2/6 and 1/3 candidates, respectively, under a six-candidate ceiling. All execute successfully, so capacity alone does not choose among them. Resource expectations and candidates are frozen before screening; three new processes confirm the selected mappings, with the second reversing order.

FWHT selects 256 threads/four columns over 128 threads/one column, giving 1.315–1.318× and 1.365–1.367× across processes. Both NTT domains retain the original mapping, timed once per process; unity denotes identity (Table V). NTT checks are full-batch exact; all twelve FWHT candidates match CPU FP32 over $2^{20}$ outputs. Fixing an internal leaf does not fix execution geometry: FWHT changes threads and columns, and NTT's shared-radix4 traversals use different wrappers. These are complete-organization outcomes, not isolated grouping effects. The construction retains original endpoints and uses measured NTT selection.

TABLE VI
INDEPENDENT SERVICES REPRODUCE COMPLETE-PLAN TIMES RETROSPECTIVELY. G COUNTS EXECUTION GROUPS; SHARED STORAGE IS IN KIB AND TIMES ARE IN MS.

| Request $(N/B)$ | G | $\text{SMEM}^{\max}$ | Retro. | Confirm |
|---|---|---|---|---|
| FFT64 $2^{20}/16$ | 2 | 64 | 1.0651 | 1.0727 |
| FWHT32 $2^{16}/16$ | 7 | 4 | 0.0450 | 0.0449 |
| FWHT32 $2^{15}/32$ | 1 | 32 | 0.0186 | 0.0184 |

**Optional mechanisms have costs.** In a factor-streamed FFT control at $N = 256, B = 16384$, slice overlap changes native/imported-core latency from 127.560/126.331 to 132.346/130.957 $\mu$s. A separate pipeline-batching control loses to bulk in all 27 comparisons; at $N = 4096, B = 64$, times are 45.636 versus 29.266 $\mu$s. Writer-aligned layout also loses in three tested pairs, whereas greater resident depth wins in three. These interventions test applicability, not whether a mechanism must help every family. Eight-candidate searches with overlap, resident fusion, or successor layout disabled show small FFT/FWHT differences and NTT differences of at most 2.9%. Two rejected unrestricted mappings leave unequal accepted pools, limiting attribution. These finite comparisons do not establish global optima.

**Composition reuse.** Across 138 coset, negacyclic, and RNS requests, replaying existing cyclic mappings per modulus gives GM 1.691× over internal public defaults (range 1.051–2.996×). Each request first geometrically averages its two input-pattern ratios. All 552 measurements across two input patterns and two implementations pass exact checks. Both core and organization may change, so this supports composition reuse rather than fixed-core attribution or joint RNS search. Default runs first, and Graph replays are not independent process trials.

### D. Analysis

**Question 4: Which resource costs explain organization choices?** The preceding controls show conditional gains. We now connect dependency-derived demand, compiled resources, and measured execution. NCU counters and NSYS timelines characterize matched internal controls; they do not independently explain gains over a different external-library implementation.

**From resource demand to complete-plan service.** Dependency analysis supplies region counts and logical interface traffic; compilation supplies storage and register requirements. Independent service curves supply timing. Table VI connects these layers for three FFT/FWHT requests, using 34 service keys and 117 clean probes. Median relative errors are 0.565% over 68 group holdouts and 0.429% over 15 complete-plan confirmations. Removing complete-plan-fitted fallback parameters leaves predictions unchanged. However, the final composition artifact postdates confirmation, so this is retrospective validation, not certified prospective prediction. It does not cover NTT.

Unlike the prospective measured choices in Table V, these estimates test composition from independent local services. The two studies test distinct links of the construction-to-execution chain; neither establishes unseen-hardware prediction.

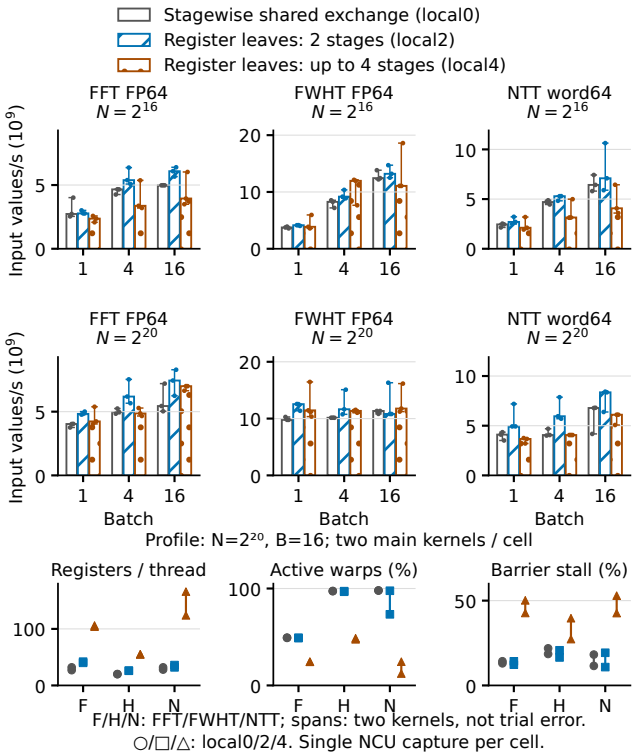



Fig. 11. Local grouping changes throughput and resource pressure. Top: medians, process points, and ranges. Bottom: two kernel records per organization at $N = 2^{20}, B = 16$; their spans are not trial uncertainty.

**More local reuse need not improve execution.** Figure 11 retains two outer bands (8+8 stages at $N = 2^{16}$ and 10+10 at $N = 2^{20}$), changing local shared exchange under fixed arithmetic. At $N = 2^{20}, B = 16$, each organization's two kernels launch 16,384 compute groups of 256 threads. Two-stage leaves change measured DRAM traffic little. Four-stage leaves increase register demand and reduce active-warp fractions from roughly 49% to 24% for FFT, 97% to 48% for FWHT, and 98% to 12–25% for NTT; barrier-stall percentages also rise. Fewer local exchanges can therefore accompany greater resource pressure and waiting. Shared-access counts and instruction-level attribution are not available for these controls.

**Overlap must offset fragmentation.** The factor-streamed FFT variants both launch three kernels, with measured L2 traffic differing by less than 0.2%. The imported core uses fewer registers, yet improves the complete call by only about 1%. NSYS observes overlap in the bulk/pipeline control, but pipeline launches eight kernels instead of two. Observed concurrency alone is not evidence of faster completion. Similarly, six stage-grouping controls reduce launches from 12 to 2 or 18 to 3; capture traffic includes setup and replay and must not be read as ordinary complete-call traffic.

**Boundary service and local-core limits.** At FP16 $N = 4096, B = 256$, matched two-band implementations retain

packed arithmetic instruction counts. Revised ownership and writeback reduce store sectors per request from 32 to 4 in both bands and total warp instructions from 34.32 to 11.48 million. Barrier-stall fractions fall from 75.2/55.8% to 10.5/14.0%. The complete call improves from 126.26 to 43.16 $\mu$s, although the 34.20 $\mu$s single-band alternative and 11.11 $\mu$s cuFFT call remain faster. These counters do not imply eightfold fewer external-memory bytes, and some shared-memory bank conflicts increase.

Load also changes the preferred organization. FP16 radix-4 single-/two-band times are 13.11/9.27 $\mu$s at $N = 4096, B = 4$ but 33.59/39.83 $\mu$s at $B = 256$; at $N = 8192, B = 32$ they are 32.36/19.35 $\mu$s. Single-band execution and unit expansion must remain available. In the independent public FP16 cohort, paired imported local transforms use one pass at $N = 4096/8192$ and two factors at larger lengths, with writer-aligned handoffs and grouped, prefetched tiles at larger batches. Factor kernels use 96–128 registers/thread and about 15–17 active warps, versus cuFFT's 32–48 registers and 35–59 warps. Resource pressure accompanies a local-core efficiency gap, although two factor passes beat cuFFT's three passes at $N = 2^{18}, B = 32$. Organization benefits and provider limitations can coexist.

**Preparation remains a separate cost.** A ten-request public DEFAULT FWHT check takes 43.2 minutes to prepare plans. CPU preparation optimizations preserve all scores and rankings across 30,146 and 371,848 candidates. Three public requests retain their mappings and outputs while preparation falls from 17.7 to 14.8 minutes. These shared-host measurements are not interleaved trials or an installation benchmark. Reuse amortizes preparation; the first-call path remains costly and is excluded from warm execution.

## VII. Related Work

Prior work provides transform generation, specialized execution, and reusable hardware mapping abstractions. We build on these foundations to connect butterfly dependencies to resource demand and executable handoffs.

**Transform generation and specialized execution.** FFTW combines codelets with empirical planning; SPIRAL/FFTX generates transforms and surrounding computation from algebraic descriptions [2], [12], [13]. VkFFT generates GPU implementations with grouped strided transforms and layout choices. GPU-NTT optimizes locality, kernel count, block size, and shape [4], [5], [9], [24]. FlashFFTConv combines tensor-core FFT decomposition, batch/hidden tiling, and memory-aware fusion [3]. HadaCore maps FWHT chunks to warps and vectors to threadblocks with shared-memory transposes [25]. These systems optimize both arithmetic and organization.

Table VII compares representative execution and placement mechanisms. These precedents make specialized organization a starting point for our study: we ask which construction and selection rules can be shared while retaining their arithmetic specialization.

**Tensor compilation and tile scheduling.** TVM separates computation from scheduling [26]; Ansor searches hierarchical schedule sketches [27]; ROLLER constructs hardware-aligned tiles [28]. TensorIR provides block signatures, outer transformations, and tensorization [29]. Welder connects producer/consumer tiles and schedules reuse through traffic and footprint estimates [30]. These establish tiling and reusable compute interfaces. For butterflies, the changing pair relation identifies the values needed by a stage group and the producers that a successor must await, before choosing a physical tile or arithmetic core. This separates dependency construction from a particular kernel implementation while preserving the operator's arithmetic, direction, ordering, and precision contract.

TABLE VII
Transform systems expose different execution and placement choices.

| System | Ops. | Work unit | Stage split | Data split | Placement |
|---|---|---|---|---|---|
| FFTW [2] | FFT, DCT/ DST | codelets | recursive plans | vector loops | buffered copies |
| SPIRAL / FFTX [12], [13] | FFT apps. | formulas / plans | algebraic rewrite | NR | generated glue |
| VkFFT [9], [24] | FFT, DCT | GPU kernels | Four-Step | grouped FFTs | reshuffled layout |
| GPU-NTT [4], [5] | NTT | CUDA kernels | Merge / Four-Step | NR | global-memory exchange |
| Flash FFTConv [3] | FFT conv. | CUDA kernels | matrix factors | batch / hidden | retain / materialize |
| HadaCore [25] | FWHT | CTA vectors | 16-point groups | warp chunks | SMEM transpose |
| cu Butterfly | FFT, NTT, FWHT | tile subgraphs | bit bands | tile groups | successor-aligned stores |

Representative mechanisms. Apps./conv.: applications/convolution; CTA: threadblock; SMEM: shared memory. NR: the partition is unresolved in this comparison, not unsupported. Matrix factors and 16-point groups describe local transforms.

**Hierarchical dataflow and positioning.** Eyeriss analyzes data movement under controlled resources; Timeloop separates workload, architecture, and hierarchical spatial/temporal mappings [19], [31]. Applying these principles to butterflies requires legal resident tile subgraphs and their remaining communication. GPU blocks also share resources, precluding an assumption of dedicated pipeline hardware.

Our distinction is the butterfly-specific connection between dependencies and resource demand. Stage-bit sets construct dependency-closed regions; their intersections derive communication and readiness (Section III). Spatial and temporal choices then characterize live state, transfers, core work, and ready work at each storage scope. Hierarchical hybrid dataflow uses this characterization to construct residency and handoffs while retaining compatible specialized cores. The general ideas of transform generation, arithmetic/schedule separation, and hierarchical dataflow have the precedents above; we do not claim that prior systems cannot express equivalent programs. Fixed-core experiments (Section VI-C) test whether the resulting organization choices improve performance independently

of arithmetic substitution.

## VIII. Conclusion

cuButterfly derives resource demands and executable handoffs from butterfly dependencies. Hierarchical hybrid dataflow uses this construction to organize compatible cores and workloads under hardware constraints. Fixed-core controls establish conditional organization gains, while library and application comparisons measure practical benefits and losses on the tested A100 workloads and numerical contracts. Cross-architecture performance remains unvalidated.

## References


[1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.

[2] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005. [Online]. Available: https://www.fftw.org/fftw-paper-ieee.pdf

[3] D. Y. Fu, H. Kumbong, E. Nguyen, and C. Ré, "FlashFFTConv: Efficient convolutions for long sequences with tensor cores," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://arxiv.org/abs/2311.05908

[4] A. Ş. Özcan and E. Savaş, "Two algorithms for fast GPU implementation of NTT," Cryptology ePrint Archive, Paper 2023/1410, 2023. [Online]. Available: https://eprint.iacr.org/2023/1410

[5] A. Ozcan, A. Javeed, and E. Savas, "High-performance number theoretic transform on GPU through radix2-CT and 4-step algorithms," *IEEE Access*, vol. 13, pp. 87 862–87 883, 2025. [Online]. Available: https://doi.org/10.1109/ACCESS.2025.3570024

[6] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa, "QuIP#: Even better LLM quantization with Hadamard incoherence and lattice codebooks," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235, 2024, pp. 48 630–48 656. [Online]. Available: https://proceedings.mlr.press/v235/tseng24a.html

[7] T. Dao, A. Gu, M. Eichhorn, A. Rudra, and C. Ré, "Learning fast algorithms for linear transforms using butterfly factorizations," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 97, 2019, pp. 1517–1527. [Online]. Available: https://proceedings.mlr.press/v97/dao19a.html

[8] NVIDIA, "cuFFT library documentation," https://docs.nvidia.com/cuda/cufft/index.html, accessed September 16, 2026; experimental binary versions are recorded separately.

[9] VkFFT Project, "VkFFT: GPU fast Fourier transform library," https://github.com/DTolm/VkFFT, official implementation and documentation; accessed September 16, 2026.

[10] GPU-NTT Project, "GPU-NTT: Merge and 4-step number theoretic transforms on GPUs," https://github.com/Alisah-Ozcan/GPU-NTT, official implementation; accessed September 16, 2026. Measured adapter versions and semantic scope are recorded separately.

[11] Dao AI Lab, "Fast Hadamard transform in CUDA, with a PyTorch interface," https://github.com/Dao-AILab/fast-hadamard-transform, official implementation; accessed September 16, 2026.

[12] M. Püschel, J. M. F. Moura, J. R. Johnson, D. Padua, M. M. Veloso, B. W. Singer, J. Xiong, F. Franchetti, A. Gačić, Y. Voronenko, K. Chen, R. W. Johnson, and N. Rizzolo, "SPIRAL: Code generation for DSP transforms," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 232–275, 2005.

[13] F. Franchetti, D. G. Spampinato, A. Kulkarni, D. T. Popovici, T. M. Low, M. Franusich, A. Canning, P. McCorquodale, B. V. Straalen, and P. Colella, "FFTX and SpectralPack: A first look," in *IEEE 25th International Conference on High Performance Computing Workshops (HiPCW)*, 2018, pp. 18–27. [Online]. Available: https://www.spiral.net/doc/papers/hipc_2018.pdf

[14] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.

[15] W. M. Gentleman and G. Sande, "Fast Fourier transforms," in *AFIPS Fall Joint Computer Conference*, 1966.

[16] T. G. Stockham, Jr., "High-speed convolution and correlation," in *AFIPS Spring Joint Computer Conference*, 1966.

[17] NVIDIA, "NVIDIA Ampere GPU Architecture Tuning Guide," https://docs.nvidia.com/cuda/ampere-tuning-guide/index.html, accessed September 16, 2026.

[18] ——, "cuFFTDx: Execution methods," https://docs.nvidia.com/cuda/cufftdx/api/methods.html, accessed September 16, 2026.

[19] A. Parashar, P. Raina, Y. S. Shao, Y.-H. Chen, V. A. Ying, A. Mukkara, R. Venkatesan, B. Khailany, S. W. Keckler, and J. Emer, "Timeloop: A systematic approach to DNN accelerator evaluation," in *IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2019, pp. 304–315.

[20] NVIDIA, "CUDA Programming Guide: Shared-memory bank swizzling," https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/async-copies.html\#shared-memory-bank-swizzling, accessed October 29, 2026.

[21] HEonGPU Contributors, "HEonGPU: Gpu-accelerated fully homomorphic encryption library," https://github.com/Alisah-Ozcan/HEonGPU, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[22] NeuralOperator Contributors, "NeuralOperator: Learning in infinite dimensions," https://github.com/neuraloperator/neuraloperator, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[23] QuaRot Contributors, "QuaRot: Outlier-free 4-bit inference in rotated LLMs," https://github.com/spcl/QuaRot, open-source implementation; accessed September 21, 2026. Application source revision is pinned separately in the experiment manifest.

[24] D. Tolmachev, "VkFFT—a performant, cross-platform and open-source GPU FFT library," *IEEE Access*, vol. 11, pp. 12 039–12 058, 2023. [Online]. Available: https://doi.org/10.1109/ACCESS.2023.3242240

[25] K. Agarwal, R. Astra, A. Hoque, M. Srivatsa, R. Ganti, L. Wright, and S. Chen, "HadaCore: Tensor core accelerated Hadamard transform kernel," arXiv:2412.08832, 2024. [Online]. Available: https://arxiv.org/abs/2412.08832

[26] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 2018, pp. 578–594. [Online]. Available: https://www.usenix.org/conference/osdi18/presentation/chen

[27] L. Zheng, C. Jia, M. Sun, Z. Wu, C. H. Yu, A. Haj-Ali, Y. Wang, J. Yang, D. Zhuo, K. Sen, J. E. Gonzalez, and I. Stoica, "Ansor: Generating high-performance tensor programs for deep learning," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 863–879. [Online]. Available: https://www.usenix.org/conference/osdi20/presentation/zheng

[28] H. Zhu, R. Wu, Y. Diao, S. Ke, H. Li, C. Zhang, J. Xue, L. Ma, Y. Xia, W. Cui, F. Yang, M. Yang, L. Zhou, A. Cidon, and G. Pekhimenko, "ROLLER: Fast and efficient tensor compilation for deep learning," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 233–248. [Online]. Available: https://www.usenix.org/conference/osdi22/presentation/zhu

[29] S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen, "TensorIR: An abstraction for automatic tensorized program optimization," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. ACM, 2023, pp. 804–817. [Online]. Available: https://arxiv.org/abs/2207.04296

[30] Y. Shi, Z. Yang, J. Xue, L. Ma, Y. Xia, Z. Miao, Y. Guo, F. Yang, and L. Zhou, "Welder: Scheduling deep learning memory access via Tile-graph," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023, pp. 701–718. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/shi

[31] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proceedings of the 43rd Annual International Symposium on Computer Architecture*, 2016, pp. 367–379. [Online]. Available: https://eems.mit.edu/wp-content/uploads/2016/04/eyeriss_isca_2016.pdf